1. Relational Clauses (Relational Algebra)

The relational model represents data in the form of tables, also called relations. Relational algebra is a procedural query language used to perform operations on relational tables.

Important relational algebra operations:

1. Selection (σ): Selects rows that satisfy a particular condition.
    - Example: Select students whose marks are greater than 80.
    - σ Marks > 80 (Student)
2. Projection (π): Selects specific columns from a table.
    - Example: Display only student names.
    - π Name (Student)
3. Union (∪): Combines records from two relations and removes duplicates.
4. Set Difference (−): Returns records present in one relation but not in another.
5. Cartesian Product (×): Combines every row of one table with every row of another table.
6. Rename (ρ): Renames a relation or its attributes.
7. Join (⋈): Combines rows from two tables based on a related column.

Types of Join:

- Inner Join
- Left Outer Join
- Right Outer Join
- Full Outer Join
- Natural Join
- Equi Join

2. Functional Dependency

Functional dependency is a relationship between attributes in a database table.

It describes how one attribute determines another attribute.

Definition: If attribute A uniquely determines attribute B, then B is functionally dependent on A.

It is represented as:

\[A \rightarrow B \]

Example:

Student_ID	Student_Name	Department
101	Rahul	BCA
102	Priya	BBA
103	Amit	BCA

Here:

Student_ID → Student_Name

This means Student_ID uniquely determines Student_Name.

Types of Functional Dependency

1. ***Trivial Dependency: When B is a subset of A.***
    - Example: (Student_ID, Name) → Name.
2. ***Non-Trivial Dependency: When B is not a subset of A.***
    - Example: Student_ID → Name.
3. ***Full Functional Dependency: An attribute depends on the complete composite key, not just a part of it.***
4. ***Partial Dependency: An attribute depends on only a part of a composite key.***
5. ***Transitive Dependency: When A determines B and B determines C, then A indirectly determines C.***

Example:

\[A \rightarrow B,\quad B \rightarrow C \]

Therefore:

\[A \rightarrow C \]

Importance:

- Reduces data redundancy.
- Helps in database normalization.
- Maintains data consistency.
- Prevents update anomalies.

3. Query Optimization Algorithms

Query optimization is the process of selecting the most efficient way to execute a database query.

The main objective is to reduce:

- Query execution time.
- CPU usage.
- Memory usage.
- Disk I/O operations.

Steps of Query Optimization:

1. Query Parsing: Checks the SQL query for syntax and validity.
2. Query Translation: Converts SQL into an internal representation.
3. Query Optimization: Finds the most efficient execution plan.
4. Query Execution: Executes the selected plan and returns results.

Important Query Optimization Techniques:

1. Heuristic Optimization

Uses predefined rules to improve query performance.

Examples:

- Perform selection operations early.
- Perform projection early.
- Replace Cartesian product followed by selection with a join.
- Apply restrictive conditions as early as possible.

2. Cost-Based Optimization

The database estimates the cost of different execution plans and selects the plan with the lowest estimated cost.

It considers:

- CPU cost.
- Memory cost.
- Disk I/O cost.
- Number of records.

3. Join Ordering

The optimizer chooses the best order to join multiple tables.

4. Indexing

Indexes help retrieve records faster without scanning the entire table.

Example:

Instead of retrieving all employee records and then filtering them:

SELECT * FROM Employee

WHERE Salary > 50000;

The database can use an index on Salary to find matching records faster.

4. Database Security

Database security refers to protecting a database from unauthorized access, misuse, modification, destruction and data theft.

Objectives of Database Security:

1. Confidentiality: Only authorized users can access sensitive information.

2. Integrity: Data must remain accurate and consistent.
3. Availability: Data should be available whenever required.
4. Authentication: Verifies the identity of users.
5. Authorization: Determines what operations a user can perform.

Database Security Techniques:

- Password protection.
- Data encryption.
- User authentication.
- Access control.
- Database backup.
- Auditing and monitoring.
- Firewalls.

5. DAC (Discretionary Access Control)

DAC stands for Discretionary Access Control.

It is a security model in which the owner of a database object decides who can access that object and what permissions they receive.

Features:

- The owner controls access permissions.
- Permissions can be granted or revoked.
- Users may share their permissions with others if allowed.

Example:

A database administrator gives a student permission to read a Student table.

GRANT SELECT ON Student TO User1;

To remove permission:

REVOKE SELECT ON Student FROM User1;

Advantages:

- Flexible.
- Easy to implement.
- Suitable for small organizations.

Disadvantages:

- Permissions may spread to unauthorized users.
- Less secure in highly sensitive environments.

6. MAC (Mandatory Access Control)

MAC stands for Mandatory Access Control.

It is a security model in which access permissions are controlled by a central authority based on security levels and classifications.

Users cannot independently change access permissions.

Features:

- Uses security classifications.
- Access is controlled by system policies.
- Users cannot freely grant permissions.
- Provides strict security.

Example:

A government database may classify information as:

- Top Secret
- Secret
- Confidential
- Public

A user can access information only if their security clearance permits it.

Advantages:

- High level of security.
- Prevents unauthorized access.
- Suitable for military and government organizations.

Disadvantages:

- Less flexible.
- Difficult to manage.
- Requires strict administration.

7. RBAC (Role-Based Access Control)

RBAC stands for Role-Based Access Control.

In this model, permissions are assigned to roles, and users are assigned those roles.

Features:

- Access depends on the user's role.
- Permissions are assigned to roles.
- Easy to manage multiple users.
- Supports organizational security policies.

Example:

Role	Permissions
Admin	Insert, Update, Delete, View
Teacher	Insert, Update, View
Student	View

If a student becomes a teacher, the administrator can assign the teacher role.

Advantages:

- Easy permission management.

- Reduces administrative work.
- Improves security.
- Suitable for large organizations.

Disadvantages:

- Role management can become complex.
- Incorrect role assignment can create security risks.

Difference Between DAC, MAC and RBAC

Basis	**DAC**	**MAC**	**RBAC**
Full form	Discretionary	Mandatory	Role-Based
Control	Object owner	Central authority	Assigned roles
Flexibility	High	Low	Medium
Security	Moderate	Very high	High
Access based on	Ownership	Security level	User role

8. SQL Injection

SQL Injection is a type of cyberattack in which an attacker inserts malicious SQL code into an input field to manipulate database queries.

It can allow attackers to:

- Access unauthorized information.
- Modify database records.
- Delete important data.
- Bypass authentication.

Example:

Suppose a login query is:

SELECT * FROM Users

WHERE username = 'admin'

AND password = '1234';

An attacker may enter specially crafted input that changes the meaning of the SQL query.

Prevention of SQL Injection:

1. Use prepared statements.
2. Use parameterized queries.
3. Validate user input.
4. Avoid directly concatenating user input into SQL queries.
5. Apply proper database permissions.
6. Keep database software updated.

Example of a safer Java approach:

String sql = "SELECT * FROM Users WHERE username = ? AND password = ?";

PreparedStatement ps = con.prepareStatement(sql);

ps.setString(1, username);

ps.setString(2, password);

ResultSet rs = ps.executeQuery();

Prepared statements separate SQL instructions from user-provided values, helping prevent SQL injection.

9. Data Warehousing

A Data Warehouse is a centralized storage system that collects data from multiple sources for reporting, analysis and decision-making.

It stores historical data and helps organizations analyze business performance.

Characteristics of a Data Warehouse:

1. Subject-Oriented: Organized around subjects such as customers, sales and products.
2. Integrated: Combines data from multiple sources.
3. Time-Variant: Stores historical data over time.
4. Non-Volatile: Stored data is generally stable and not frequently changed or deleted.

Components:

- Data Sources: Collect data from different databases and systems.
- ETL: Extract, Transform and Load data.
- Data Warehouse: Stores processed data.
- OLAP: Performs multidimensional analysis.
- Reporting Tools: Generate reports and dashboards.

Advantages:

- Helps in decision-making.
- Stores historical information.
- Improves business analysis.
- Combines data from different sources.

10. Data Mining

Data Mining is the process of discovering useful patterns, relationships and hidden information from large amounts of data.

It uses statistical techniques, machine learning and database technologies to analyze data.

Example:

A supermarket analyzes customer purchases and discovers that people who buy bread often buy butter.

The supermarket can use this information for product placement and marketing.

Steps of Data Mining:

1. Data Cleaning: Removes incorrect or duplicate data.

2. Data Integration: Combines data from multiple sources.
3. Data Selection: Selects relevant data.
4. Data Transformation: Converts data into a suitable format.
5. Data Mining: Finds patterns and relationships.
6. Pattern Evaluation: Identifies useful patterns.
7. Knowledge Presentation: Presents results using reports and visualizations.

Data Mining Techniques:

1. Classification: Groups data into predefined categories.
2. Clustering: Groups similar data without predefined categories.
3. Association: Finds relationships between items.
4. Prediction: Predicts future outcomes.
5. Regression: Identifies relationships between variables.
6. Anomaly Detection: Identifies unusual patterns.

Applications:

- Banking and finance.
- Healthcare.
- Retail business.
- Fraud detection.
- Customer analysis.
- Education.